

UNIVERSITÀ POLITECNICA DELLE MARCHE

FACOLTÀ DI INGEGNERIA

Dipartimento di Ingegneria dell'Informazione

Corso di Laurea Magistrale in Ingegneria Informatica e dell'Automazione



Bert

Autori del progetto

Erxhes Dedja — 1119284

Docenti

Prof. Domenico Ursino

ANNO ACCADEMICO 2024-2025

Indice

1	INTRODUZIONE	1
1.1	NLP	1
1.2	BERT	2
2	Dataset	4
2.1	Introduzione	4
2.2	Struttura Del Dataset	4
3	Modello	6
3.1	Fase di ETL	6
3.2	Sviluppo Modello	8
3.3	Addestramento Modello	11
4	Test	13
4.1	Introduzione	13
4.2	Metriche	13
4.3	Matrice di confusione	14
	Bibliografia	16

Elenco delle figure

1.1	Funzionamento NLP	2
1.2	Pre-training e Fine-tuning BERT	3
2.1	Le prime 10 righe del dataset	5
3.1	Eliminazione dei duplicati	6
3.2	Numero totale di righe	6
3.3	Randomizzazione file	7
3.4	Split del file csv	8
3.5	Esempio Tokenizzazione	9
3.6	Esempio Tokenizzazione dei vettori	10
3.7	Confronto tra training e validation loss	12
3.8	Validation accuracy	12
4.1	Matrice di confusione	15

Capitolo 1

INTRODUZIONE

In questo capitolo l'obiettivo principale è quello di fornire una panoramica chiara e comprensibile del contesto teorico alla base dell'elaborazione del linguaggio naturale e dell'architettura profonda utilizzata nel progetto.

1.1 NLP

Il natural language processing (NLP) è una branca dell'intelligenza artificiale che si occupa dell'interazione tra computer e linguaggio umano. L'obiettivo principale è far sì che i computer possano capire, creare e rispondere al linguaggio umano in modo significativo. NLP permette ai computer di interagire con le persone nella loro lingua e di svolgere compiti legati al linguaggio. Ad esempio, consente dei computer di comprendere testi, ascoltare discorsi, analizzare il significato, valutare i sentimenti e individuare le parti più rilevanti.

Per quanto concerne i compiti operativi dell'ingegneria del linguaggio, le principali applicazioni dell'NLP si articolano nei seguenti punti:

- **Classificazione del testo:** Identificare la categoria di un testo, ad esempio classificare email come "spam" o "non spam".
- **Analisi del Sentiment:** Consente di analizzare le opinioni degli utenti e di stabilire se un testo esprime un orientamento positivo o negativo.
- **Chatbot e assistenti virtuali:** Sistemi interattivi quali Alexa, Siri, Google Assistant e interfacce automatiche per il supporto clienti.
- **Traduzione Automatica:** Tradurre in modo dinamico il testo da una lingua all'altra, come ad esempio l'applicazione Google Translate.
- **Riconoscimento vocale:** Modelli strutturati che convertono il parlato in testo, utile per prendere appunti, creare documenti e trascrivere conversazioni.
- **Correzione grammaticale:** Algoritmi che migliorano la grammatica e la coerenza del testo scritto, fornendo suggerimenti automatici come Grammarly.

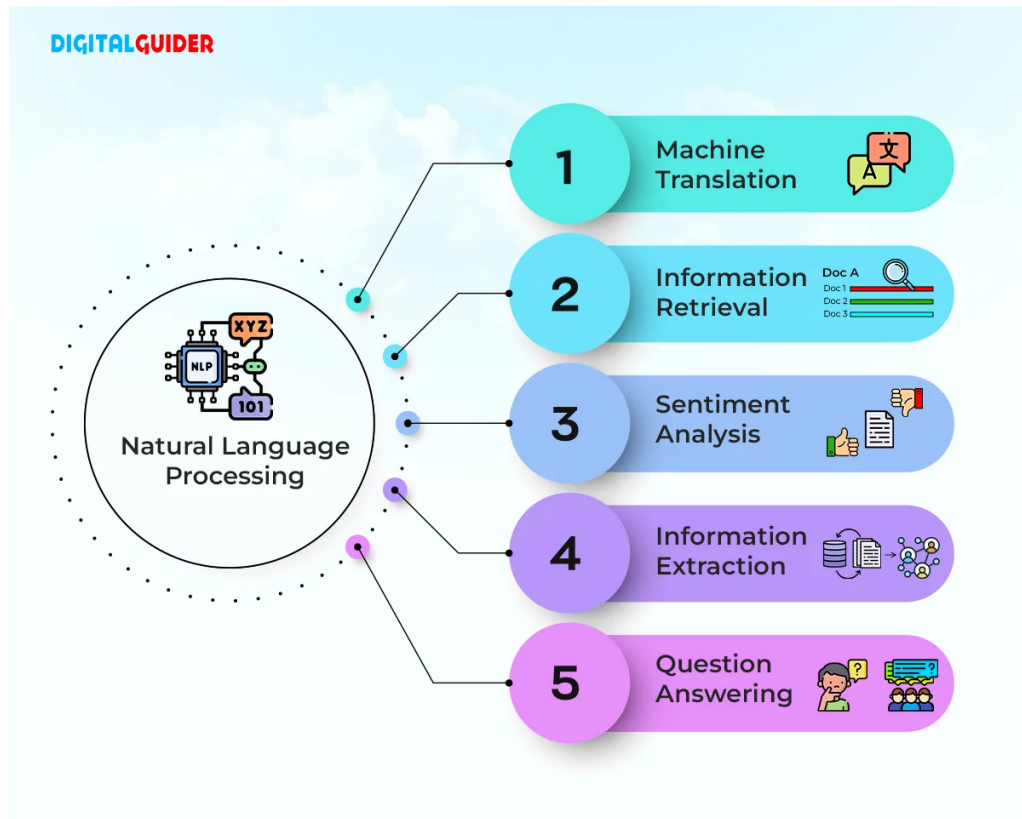


Figura 1.1: Funzionamento NLP

1.2 BERT

Il sistema BERT, acronimo di Bidirectional Encoder Representations from Transformers, è un framework avanzato per l'elaborazione del linguaggio naturale (NLP) basato sul concetto di reti neurali profonde. Introdotto nel 2018, BERT ha rivoluzionato il modo in cui i modelli comprendono il linguaggio, stabilendo nuovi standard di precisione in una vasta gamma di compiti linguistici.

L'importanza di questo algoritmo risiede nella sua capacità di analizzare le parole in modo bidirezionale, ossia considerando ogni parola nel contesto delle parole che la precedono e che la seguono. Questo approccio permette a BERT di comprendere il significato e l'intento alla base di una chiave di ricerca in maniera avanzata.

Dal punto di vista structural, BERT si basa su un'architettura chiamata Transformer, introdotta per la prima volta nel 2017. I Transformer usano una componente chiamata Self-Attention, che permette al modello di concentrarsi sulle parti più rilevanti del testo mentre lo elabora.

L'addestramento preliminare del modello si fonda stabilmente su due compiti principali:

- **Masked Language Model (MLM):** È una tecnica che consente a BERT di imparare il significato delle parole nel contesto in cui si trova. Una percentuale dei token (parole) in ciascuna frase viene mascherata a caso. Il compito del modello è predire i token mascherati basandosi sul contesto circostante. Ad esempio, nella frase "Il

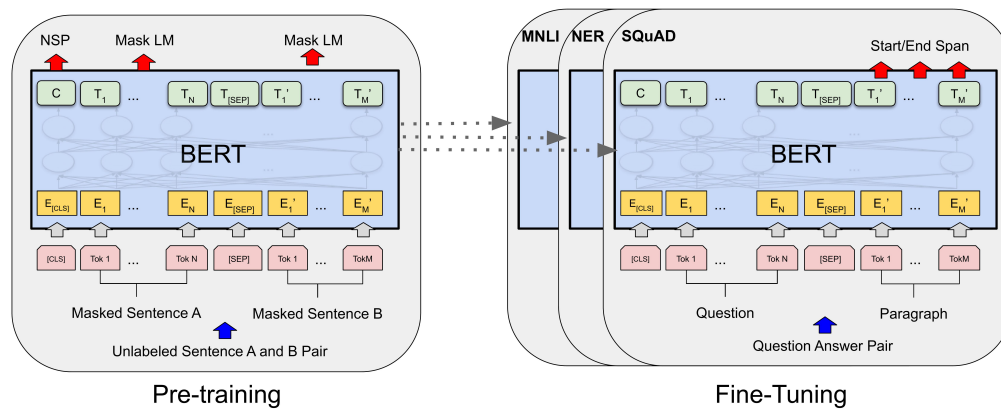


Figura 1.2: Pre-training e Fine-tuning BERT

giocatore ha preso un [MASK] rosso", BERT deve imparare a prevedere che il token mascherato è "cartellino".

- **Next Sentence Prediction (NSP):** Questa tecnica aiuta BERT a comprendere le relazioni tra due frasi consecutive. In questa fase, il modello riceve coppie di frasi e deve determinare se la seconda frase segue logicamente la prima nel testo originale. Ad esempio, date le frasi "Il giocatore ha commesso un fallo" e "Il giocatore ha preso un cartellino rosso", BERT deve decidere se la seconda frase è una continuazione della prima o meno.

Capitolo 2

Dataset

2.1 Introduzione

Il dataset è reperibile direttamente su [Kaggle](#). Contiene recensioni di film prese dal database di IMDB, classificate per sentiment.

2.2 Struttura Del Dataset

Il dataset è composto da 40.000 righe e 2 colonne. Esso rappresenta recensioni di film accompagnate da etichette di sentiment che indicano se la recensione è positiva o negativa.

Il dataset è suddiviso in due colonne:

- **Review:**

- Contiene il testo della recensione.
- È una stringa di lunghezza variabile.
- Sono presenti **39.723 valori univoci**, il che implica la presenza di alcune recensioni duplicate.
- Il numero di recensioni duplicate è pari a **277**.

- **Sentiment:**

- Indica il sentiment della recensione con valori binari:
 - * **0**: Sentiment negativo.
 - * **1**: Sentiment positivo.
- La distribuzione delle etichette è quasi perfettamente bilanciata:
 - * **20.019** recensioni con etichetta **0** (negative).
 - * **19.981** recensioni con etichetta **1** (positive).

	text	label
0	I grew up (b. 1965) watching and loving the Th...	0
1	When I put this movie in my DVD player, and sa...	0
2	Why do people who do not know what a particula...	0
3	Even though I have great interest in Biblical ...	0
4	Im a die hard Dads Army fan and nothing will e...	1
5	A terrible movie as everyone has said. What ma...	0
6	Finally watched this shocking movie last night...	1
7	I caught this film on AZN on cable. It sounded...	0
8	It may be the remake of 1987 Autumn's Tale aft...	1
9	My Super Ex Girlfriend turned out to be a plea...	1

Figura 2.1: Le prime 10 righe del dataset

Capitolo 3

Modello

3.1 Fase di ETL

Per garantire la qualità del dataset, è stato effettuato un controllo per identificare ed eliminare i record duplicati nella colonna `text`. Al termine del processo di filtraggio, il dataset è risultato pulito con un totale di **39.723 righe complessive**. La distribuzione finale delle etichette (19.815 negative e 19.908 positive) conferma il mantenimento di un perfetto bilanciamento tra le classi.

```
import pandas as pd

# Carica il file CSV
df = pd.read_csv("C:/Users/MrWhi/OneDrive/Desktop/univpm/DataScience/BertProg/movie.csv")

# Rimuovi i duplicati basati sulla colonna 'text'
df_cleaned = df.drop_duplicates(subset=['text'], keep='first')

# Verifica il numero di righe rimanenti
print(f"Numero di righe dopo la rimozione dei duplicati: {df_cleaned.shape[0]}")

# Salva il nuovo dataset pulito
df_cleaned.to_csv("movie_cleaned.csv", index=False)
```

Figura 3.1: Eliminazione dei duplicati

Il numero di righe dopo la rimozione dei duplicati:

```
[5] from collections import Counter
      Counter(df.label)

Counter({0: 19815, 1: 19908})
```

Figura 3.2: Numero totale di righe

Il dataset contiene adesso 19.815 recensioni con etichetta 0 (negativo) e 19.908 recensioni con etichetta 1 (positivo).

Per migliorare l'addestramento del modello, il dataset è stato randomizzato (shuffling) utilizzando un seme fisso (`random_state=42`) per garantire la riproducibilità. La randomizzazione è fondamentale per evitare che il modello memorizzi l'ordine dei dati, riducendo il rischio di overfitting e migliorando la generalizzazione su dati non visti.

	text	label
26596	The very first time I heard of Latter Days was...	1
39584	Millie is a sap. She marries a rich guy named ...	0
32929	Jay Chou plays an orphan raised in a kung fu s...	1
8578	I don't know what that other guy was thinking....	0
841	I grew up in Royersford, Pa. The town where Je...	1

Figura 3.3: Randomizzazione file

Successivamente, viene introdotta la funzione **cleanText**, sviluppata per normalizzare il corpus testuale attraverso una serie di operazioni di pulizia mirate, preservando al contempo il contesto semantico fondamentale per un modello basato su Transformer.

Le operazioni applicate includono:

- **Rimozione dei tag HTML:** Eliminazione di qualsiasi marcatore o tag HTML presente nel testo grezzo delle recensioni.
- **Rimozione degli URL:** Cancellazione di link web che non forniscono valore lessicale.
- **Conversione in lettere minuscole:** Trasformazione di tutto il testo in minuscolo per uniformare il vocabolario ed evitare la duplicazione dei token basata esclusivamente sul *case*.

Nota metodologica: A differenza dei classici approcci di text mining tradizionali, in questa fase **non sono state rimosse la punteggiatura e le stopwords**. Essendo BERT un modello contestuale e bidirezionale, la struttura sintattica (inclusi punti, virgole e congiunzioni) fornisce informazioni vitali sul sentiment e sull'orientamento emotivo della frase; la loro rimozione avrebbe privato il meccanismo di *Self-Attention* di connessioni semantiche cruciali.

3.2 Sviluppo Modello

Per lo sviluppo del modello abbiamo suddiviso il dataset in due parti: una per l'addestramento del modello e l'altra per il test. La parte dedicata all'addestramento contiene i dati utilizzati per istruire il modello nel riconoscimento di schemi e nella formulazione di previsioni. La parte di test, invece, è usata per valutare le prestazioni del modello su dati che non ha mai visto prima, permettendoci di misurare la sua accuratezza e capacità di generalizzazione. Questa separazione è cruciale per garantire che il modello non sia troppo adattato ai dati di addestramento, ma possa funzionare bene anche con nuovi dati.

```
# Mescola il dataset (shuffle)
df = df.sample(frac=1, random_state=42).reset_index(drop=True)
print("Dataset shuffled.")

# Divisione in train e test (80-20, stratificata per label)
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42, stratify=df['label'])
print('Train length:', len(train_df))
print(train_df.groupby(['label'])['text'].count())
print('Test length:', len(test_df))
print(test_df.groupby(['label'])['text'].count())

# Definizione delle sentences e labels per il train
sentences = train_df['text'].tolist()
labels = train_df['label'].tolist()
```

Dataset shuffled.
Train length: 31778
label
0 15852
1 15926
Name: text, dtype: int64
Test length: 7945
label
0 3963
1 3982
Name: text, dtype: int64

Figura 3.4: Split del file csv

Il dataset è stato suddiviso in due parti principali: il set di addestramento e il set di test. La divisione è stata effettuata in modo da destinare l'80% del dataset originale al set di addestramento e il restante 20% al set di test. La suddivisione è stata progettata per garantire che il modello venga allenato su una porzione significativa dei dati e testato su una porzione separata per valutare le sue prestazioni.

Il set di addestramento, costituito dall'80% del dataset originale, comprende un totale di 31.778 righe. Questo set è utilizzato per addestrare il modello, permettendogli di apprendere i pattern e le relazioni presenti nei dati. La distribuzione delle etichette all'interno del set di addestramento è ben bilanciata:

- Etichetta 0 (Negativo): 15.852 righe
- Etichetta 1 (Positivo): 15.926 righe

Questo equilibrio tra le classi è essenziale per evitare bias durante l'addestramento e garantire che il modello apprenda a riconoscere correttamente sia le recensioni positive che quelle negative.

Il set di test, costituito dal restante 20% del dataset originale, include 7.945 righe. Questo set viene utilizzato per valutare le prestazioni del modello su dati che non ha mai

visto prima, assicurando una stima accurata della sua capacità di generalizzare. Anche qui, la distribuzione delle etichette è ben bilanciata:

- Etichetta 0 (Negativo): 3.963 righe
- Etichetta 1 (Positivo): 3.982 righe

Un'operazione importante da svolgere è la tokenizzazione che consiste nel dividere un testo in unità più piccole chiamate token. Questi token possono essere parole, sottoparti di parole, caratteri o simboli, a seconda del tipo di tokenizzazione utilizzata. Una volta che il testo è tokenizzato, ogni token viene convertito in un ID numerico e inviato al modello BERT, che lo utilizza per eseguire la predizione del sentiment, la traduzione, o altre attività di linguaggio naturale.

La tokenizzazione quindi è un passo fondamentale per "preparare" il testo affinché possa essere compreso e processato da un modello di deep learning come BERT. I modelli di linguaggio, come BERT, non lavorano direttamente con il testo grezzo (parole o frasi), ma con rappresentazioni numeriche di questi token.

Nella Figura 3.5 vediamo un esempio pratico di tokenizzazione applicato a una recensione del nostro dataset. La tabella illustra come la frase originale venga suddivisa nei rispettivi token e come a ciascuno di essi venga assegnato un ID numerico univoco, corrispondente alla sua posizione nel vocabolario pre-addestrato di BERT.

Original: there are some elements that save this movie from being a total catastrophe, but are overshadowed with bad acting, plot holes,

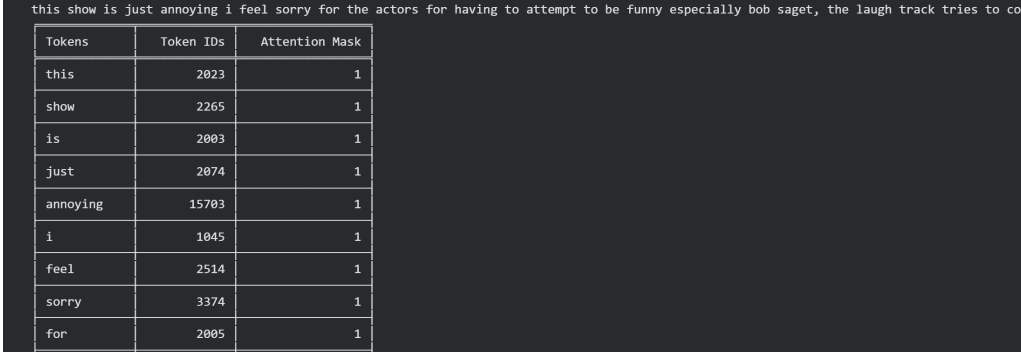
Tokens	Token IDs
there	2045
are	2024
some	2070
elements	3787
that	2008
save	3828
this	2023
movie	3185
from	2013

Figura 3.5: Esempio Tokenizzazione

Poiché BERT ha un limite massimo per il numero di token che può elaborare in una volta, è cruciale impostare un limite di troncamento stabile basato sulla lunghezza media dei testi analizzati. Per ogni frase, viene chiamato direttamente il tokenizer di Hugging Face passando il testo come parametro. Questa funzione è utilizzata per:

- Tokenizzare la frase (cioè dividerla in parole o sottoparti di parole).
- Aggiungere i token speciali [CLS] e [SEP] necessari per i modelli BERT. [CLS] viene utilizzato come token di inizio della sequenza, mentre [SEP] è usato come separatore.
- Impostare una lunghezza massima di 256 token. Se una frase supera questa lunghezza, verrà troncata alla lunghezza massima consentita (`max_length=256`, `truncation=True`).

- Creare una maschera di attenzione (attention mask) che indica quali token sono reali e quali sono padding, per evitare che il modello presti attenzione ai token aggiunti artificialmente.
- Restituire i risultati direttamente come tensori PyTorch (`return_tensors='pt'`), pronti per essere usati nei modelli di deep learning.



Tokens	Token IDs	Attention Mask
this	2023	1
show	2265	1
is	2003	1
just	2074	1
annoying	15703	1
i	1045	1
feel	2514	1
sorry	3374	1
for	2005	1

Figura 3.6: Esempio Tokenizzazione dei vettori

Dopo aver completato il pre-processamento dei dati del set di addestramento, trasformandoli in un dataset di tensori, è stato essenziale separare questi dati in due parti: il set di addestramento e il set di validazione. Il set di validazione viene utilizzato per monitorare le prestazioni del modello durante l'addestramento e per ottimizzare i suoi iperparametri. Per garantire un'efficace valutazione del modello, l'80% dei dati codificati è stato assegnato al set di addestramento, mentre il rimanente 20% è stato destinato al set di validazione (pari a 6.356 campioni).

3.3 Addestramento Modello

Per l'addestramento del modello sono stati scelti i seguenti parametri:

- **epoche:** 4
- **epsilon:** 1×10^{-8}
- **learning rate:** 2×10^{-5}
- **batch:** 16

È stato implementato un criterio di early stopping con una pazienza di 2 epoche per fermare l'addestramento se l'accuratezza di validazione non migliorava, riducendo il rischio di overfitting. I log dettagliati delle metriche e dei tempi di calcolo registrati durante il processo sono riassunti nella Tabella 3.1.

Epoca	Training Loss	Valid. Loss	Valid. Accur.	Valid. F1	Tempo Train	Tempo Valid.
1	0.3525	0.2420	0.9095	0.9041	0:19:58	0:01:37
2	0.2214	0.3158	0.9048	0.9015	0:20:02	0:01:36
3	0.1781	0.3046	0.9205	0.9176	0:20:02	0:01:35
4	0.1465	0.3315	0.9216	0.9179	0:20:01	0:01:37

Tabella 3.1: Statistiche di addestramento e validazione del modello BERT per epoca.

La training loss è diminuita costantemente lungo le quattro epoche, passando da un valore iniziale di **0.3525** (Epoca 1) fino a raggiungere **0.1465** (Epoca 4), a dimostrazione di un efficace e costante apprendimento dei pattern sul set di addestramento.

La validation loss ha toccato il suo punto di minimo assoluto già alla prima epoca con un valore pari a **0.2420**. Nelle epoche successive si è osservata una leggera e fisiologica fluttuazione (**0.3158** all'epoca 2 e **0.3046** all'epoca 3), per poi risalire leggermente fino a **0.3315** alla quarta epoca. Questo andamento evidenzia la straordinaria rapidità di convergenza tipica dei modelli pre-addestrati come BERT, che si specializzano sul task specifico già nei primissimi passaggi. La Figura 3.7 confronta l'andamento delle curve di training e validation loss.

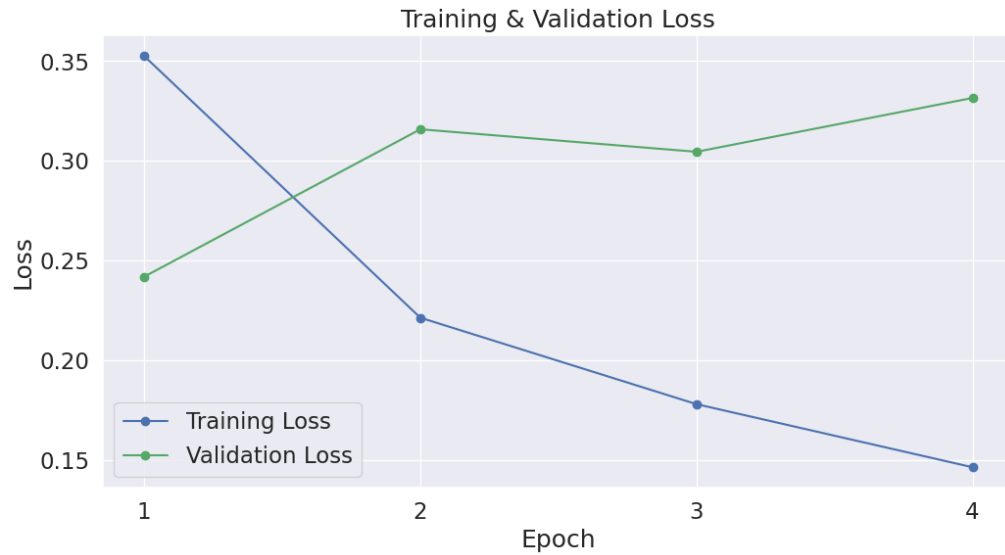


Figura 3.7: Confronto tra training e validation loss

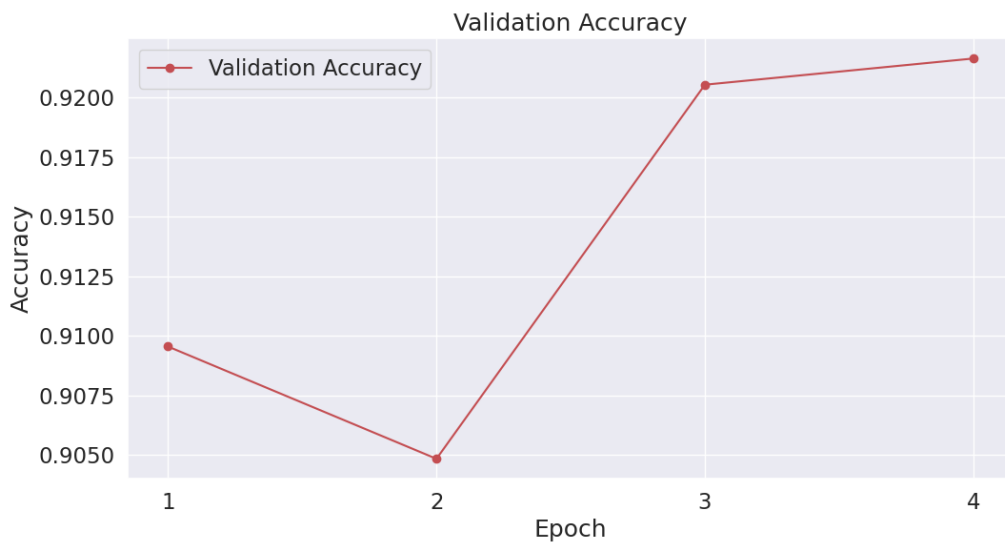


Figura 3.8: Validation accuracy

L'accuratezza di validazione, mostrata nella Figura 3.8, ha mostrato un trend incrementale solido: partendo da un ottimo **0.9095** (Epoca 1), ha superato la soglia del 92% a partire dalla terza epoca (**0.9205**), stabilizzandosi infine sul valore massimo di **0.9216** all'epoca 4. Di pari passo, l'F1-Score di validazione è cresciuto in modo coerente, consolidandosi intorno allo **0.9179**. Grazie all'implementazione del salvataggio del modello basato sul miglioramento dell'accuratezza, i pesi finali salvati corrispondono al momento di massima capacità predittiva e generalizzazione della rete.

Capitolo 4

Test

4.1 Introduzione

Per valutare le prestazioni del modello che abbiamo addestrato, utilizziamo ora i dati di test che avevamo separato in precedenza dai set di training e validazione. Proprio come abbiamo fatto con i dati di training, abbiamo tokenizzato anche i dati di test per ottenere le attention masks necessarie.

Dopo aver preparato i dati, procediamo a testare il modello e confrontiamo le etichette predette con quelle reali. Questo ci permette di misurare l'accuratezza del nostro modello e capire quanto bene riesca a fare previsioni corrette.

4.2 Metriche

Le metriche utilizzate per la valutazione quantitativa delle performance sul test set sono l'Accuracy, la Precision, la Recall e l'F1-Score.

Metrica	Formula	Descrizione
Accuracy	$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$	Percentuale di campioni correttamente classificati.
Precision	$Precision = \frac{TP}{TP+FP}$	Percentuale di predizioni positive corrette rispetto al totale delle predizioni positive.
Recall	$Recall = \frac{TP}{TP+FN}$	Capacità del modello di individuare tutti i campioni positivi (Sensibilità).
F1-Score	$F1-Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$	Media armonica tra Precision e Recall, utile per bilanciare falsi positivi e falsi negativi.

Tabella 4.1: Metriche di valutazione con relative formule

Sottoponendo il set di test (completamente inedito al modello) alla valutazione finale, BERT ha dimostrato una straordinaria capacità di generalizzazione, registrando un'Accuracy globale del **92.51%** e un **F1-Score del 92.69%**.

Il report di classificazione dettagliato mostra un comportamento estremamente bilanciato ed efficiente tra le due classi analizzate:

- **Classe Negativa (0)**: Precision pari a **0.94** e Recall pari a **0.90** (F1-Score: 0.92).
- **Classe Positiva (1)**: Precision pari a **0.91** e Recall pari a **0.95** (F1-Score: 0.93).

4.3 Matrice di confusione

La matrice di confusione è uno strumento utilizzato per valutare le prestazioni di un modello di classificazione, mostrando il confronto puntuale tra le predizioni effettuate dal modello e i valori reali in termini di frequenze.

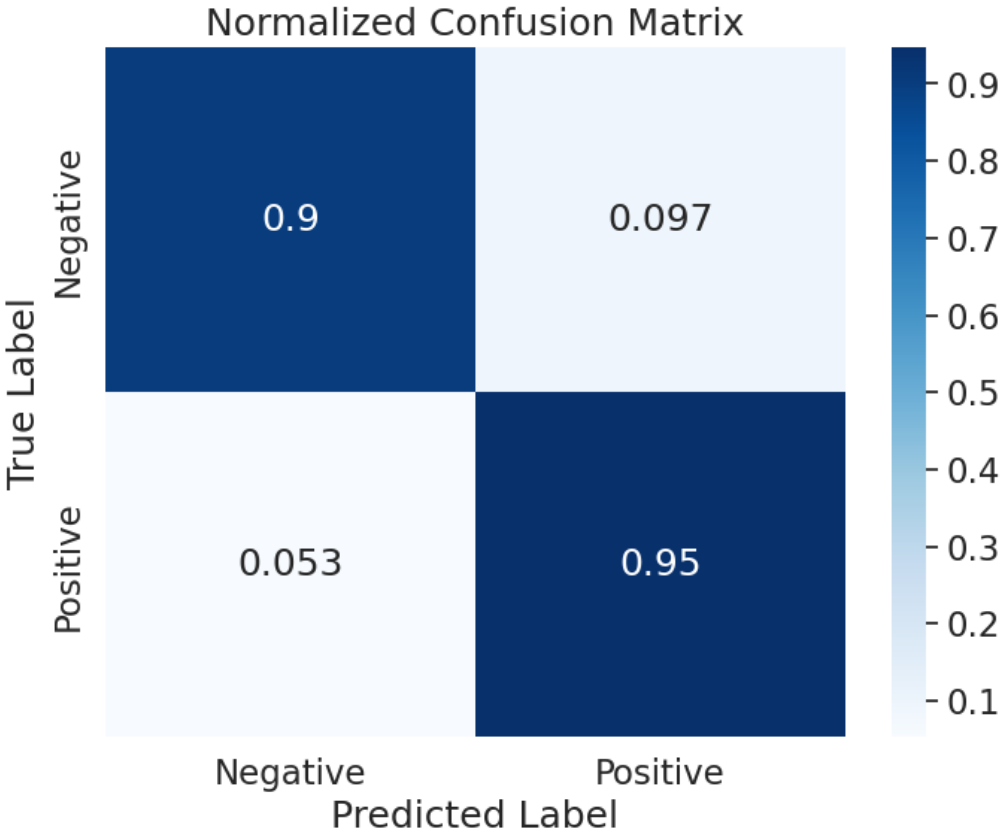


Figura 4.1: Matrice di confusione

La matrice di confusione, mostrata nella Figura 4.1, conferma la solidità del modello attraverso i volumi assoluti delle predizioni: su un totale di 7.945 recensioni di test, la diagonale principale cattura la stragrande maggioranza dei campioni con **3.580 Veri Negativi (TN)** e **3.770 Veri Positivi (TP)**.

Gli errori di classificazione si mantengono estremamente ridotti: i Falsi Positivi (FP) si attestano a 383 unità, mentre i Falsi Negativi (FN) sono appena 212. La matrice normalizzata evidenzia che il modello dimostra un’elevata efficacia nel categorizzare correttamente le recensioni positive (raggiungendo una Recall del 95%), mantenendo un’ottima stabilità ed equilibrio anche sulla classificazione del sentiment negativo (90%).

Bibliografia

Siti web consultati

- *DigitalGuider*. (2024, Dicembre 18). Tratto da UltiMaker – <https://digitalguider.com/blog/what-is-google-nlp/>
- *towardsdatascience.com* (2024, Dicembre 18). Tratto da Rinnovabili – <https://towardsdatascience.com/what-exactly-happens-when-we-fine-tune-bert-f5dc32885d76>